Windows Dev Setup (No Docker)

A lighter local setup for day-to-day development on Windows — same ten modules, no containers, using a Python virtual environment instead. Use this for fast iteration; use the Docker Compose runbook (first tab) for the full-stack demo and anything you'll show judges.

Consider WSL2 instead: `pgvector` and some native-build Python packages are noticeably easier to install under WSL2 (Windows Subsystem for Linux) than on native Windows, since WSL2 gives you a real Ubuntu userland — at that point the macOS tab's Linux/Homebrew-style commands mostly apply directly (swap `brew` for `apt`). The steps below are for native Windows (PowerShell/cmd), for anyone who'd rather not set up WSL2.

1. Prerequisites (native Windows)

1. **Python 3.11** — install from python.org (check "Add python.exe to PATH" during install) or `winget install Python.Python.3.11`.
2. **PostgreSQL 16** — install from postgresql.org's EDB installer or `winget install PostgreSQL.PostgreSQL.16`. Remember the password you set for the `postgres` superuser during install — you'll need it below.
3. **pgvector for Windows** — not bundled with the installer; see section 4 for how to add it, since it needs a matching precompiled release or a Visual Studio build.
4. **Git** — git-scm.com or `winget install Git.Git`.
5. An **Anthropic API key** (see the Docker tab's note on alternatives if you don't have one yet).

Use **PowerShell** for every command below unless noted otherwise — Command Prompt works too but the activation syntax differs (called out in section 3).

2. Clone the repo and create an isolated venv

```
git clone <repo-url>; cd fcrm-workbench

# Create the virtual environment (once)
py -3.11 -m venv .venv

# Activate it (every new terminal session) — PowerShell:
.venv\Scripts\Activate.ps1

# Command Prompt (cmd.exe) instead:
# .venv\Scripts\activate.bat

# Confirm you're using the venv's Python
where python
# → ...\fcrm-workbench\.venv\Scripts\python.exe
```

If PowerShell refuses to run the activation script (`running scripts is disabled on this system`), run this once per machine as an administrator: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

To leave the venv later: `deactivate`. `.venv\` should already be in `.gitignore` — never commit it.

3. Install dependencies into the venv

With the venv activated (`(.venv)` showing in your prompt):

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Same `requirements.txt` as the Docker image and the macOS tab — one shared dependency list keeps behavior identical across every setup. `psycopg2-binary` ships prebuilt wheels for Windows, so it normally installs without needing a compiler; if it fails, installing Microsoft C++ Build Tools resolves most native-extension build errors on Windows.

4. Set up local Postgres with pgvector

The EDB Windows installer starts PostgreSQL as a Windows service automatically, so it's already running after install — confirm with `Get-Service postgresql*`.

```
# Add Postgres's bin folder to this session's PATH (adjust the version folder
if different)
$env:Path += ";C:\Program Files\PostgreSQL\16\bin"

# Create the database and a user matching the other tabs' defaults
createdb -U postgres fcrm_workbench
psql -U postgres -d fcrm_workbench -c "CREATE USER fcrm WITH PASSWORD
'localdev';"
psql -U postgres -d fcrm_workbench -c "GRANT ALL PRIVILEGES ON DATABASE
fcrm_workbench TO fcrm;"
```

pgvector isn't in the EDB installer, so add it separately — the easiest path on Windows is a precompiled release rather than building from source:

1. Download the Windows build matching your Postgres version from the pgvector releases page (look for a Windows/ `.zip` asset; if none matches PG 16 exactly, use the source-build path in step 2).
2. If no precompiled Windows build is available for your version, build it with Visual Studio's native tools:

   ```
   git clone --branch v0.7.4 https://github.com/pgvector/pgvector.git
   cd pgvector
   # Run from a "x64 Native Tools Command Prompt for VS" so nmake/cl are on
   PATH
   nmake /F Makefile.win PG_CONFIG="C:\Program
   Files\PostgreSQL\16\bin\pg_config.exe"
   nmake /F Makefile.win install PG_CONFIG="C:\Program
   Files\PostgreSQL\16\bin\pg_config.exe"
   ```

3. Either way, confirm `vector.control` and the `vector--*.sql` files landed in `C:\Program Files\PostgreSQL\16\share\extension\`, and `vector.dll` in `C:\Program Files\PostgreSQL\16\lib\`.

4. Restart the Postgres service (`Restart-Service postgresql-x64-16`) and enable the extension:

   ```
   psql -U postgres -d fcrm_workbench -c "CREATE EXTENSION IF NOT EXISTS vector;"
   ```

5. Run the shared schema: `psql -U fcrm -d fcrm_workbench -f infra\init-db\001_schema.sql`

If this gets painful, it's the strongest single argument for using WSL2 instead (see the note at the top of this tab) — pgvector installs the same way as on the macOS/Linux tabs there.

5. Local `.env` configuration

Same as the macOS tab — `localhost` instead of the Docker Compose service name `db`:

```
# .env (local dev)
ANTHROPIC_API_KEY=sk-ant-...
DATABASE_URL=postgresql://fcrm:localdev@localhost:5432/fcrm_workbench

# Leave these blank for local dev — see section 7 below
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
```

Create this as a plain UTF-8 text file named exactly `.env` in the repo root — Notepad will happily save it as `.env.txt` unless you pick "All Files" in the save-as type dropdown.

6. Run the app locally

With the venv active and Postgres running:

```
uvicorn app.orchestrator.main:app --reload --port 8000
```

Visit `http://localhost:8000/docs` for the API docs. Run a single module's tests while working on it:

```
pytest tests/ -k extraction
```

If `uvicorn` isn't found, make sure the venv is active (step 2) — it installs into `.venv\Scripts\` and won't be on PATH otherwise.

7. Eval/observability tracing without a local Langfuse container

Same options as the macOS tab, since Langfuse's server still needs Docker either way:

- **Skip tracing locally** — leave the Langfuse keys blank in `.env`; guard those calls in code so the app runs fine without them.
- **Point at Langfuse Cloud instead** — a free hosted project needs no local infrastructure at all.
- **Run the golden-case regression suite directly** with `pytest tests/eval_regression.py` — Ragas/DeepEval scoring doesn't need Langfuse, only tracing/dashboards do.

8. When to switch back to the Docker stack

This venv setup is for fast, individual iteration on one module at a time. Switch to the Docker Compose runbook (first tab) before:

- Testing M0's orchestration across all ten modules together.
- Anything touching Langfuse tracing you'll show in the submission.
- Final integration testing, demo rehearsal, and the actual submission — judges should see the containerized, infrastructure-as-code version.

All three setups (Docker, macOS venv, Windows venv) read the same `requirements.txt`, the same `infra\init-db\001_schema.sql`, and the same module code under `app\` — nothing here forks the codebase, only how it's run locally.